safe-oracle

Technical Architecture

Planned Features & System Roadmap

From a hardened library to a production oracle-safety layer for Stellar DeFi.

Build Award — Open Track

June 2026

Built on Stellar · Powered by Soroban Smart Contracts

Contents

1 Introduction

`safe-oracle` is a defensive oracle-validation library for Soroban—a `SafeERC20` for price feeds. It wraps a SEP-40 oracle call (Reflector and compatible providers) with on-chain guardrails and returns a validated price or a typed rejection, so a DeFi protocol never acts on a manipulated, stale, or thinly-traded feed. The companion *Architecture Whitepaper* documents the system as it exists today; this document describes what we will build with Build Award funding and how the work maps to three funded milestones, the last of which is mainnet launch.

1.1 Where We Are Today

The current testnet system already ships the full guardrail core: a stateless `safe-oracle` library with five guardrails (Layer 1: deviation, staleness, cross-source; Layer 2: liquidity, thin-sampling) and an optional circuit breaker; an oracle-agnostic `oracle-validator` contract; an attested `liquidity-registry`; and an off-chain `oracle-watch` daemon with five webhook sinks. The defaults are empirically validated against three real oracle-manipulation exploits (≈ $245M rejected) and exercised live on testnet (321 tests passing, with on-chain adversarial and staleness scenarios).

1.2 What Remains for Production

Three gaps separate the current system from a protocol that other Stellar DeFi teams can adopt on mainnet:

1. **Trust hardening** of the Layer 2 attestation path (key management, multi-attester quorum) and per-asset configurability.
2. **Aggregation depth** (TWAP / SMA / EMA on top of the two-point deviation check, as endorsed by the Reflector lead developer) and a real reference integration with a protocol from the SCF Integration List, plus a public SDK and documentation.
3. **Mainnet deployment** with live Reflector, 24/7 monitoring, and published production metrics.

These map directly to Phases 1–3 below.

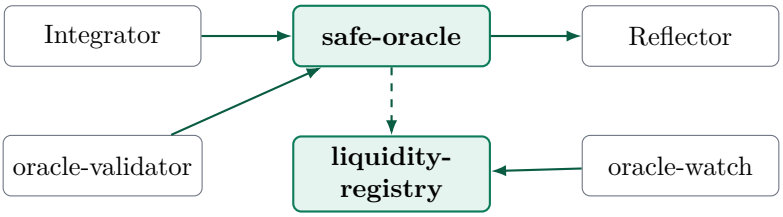

Figure 1: Current component stack. Roadmap work hardens, deepens, and ships these to mainnet rather than adding new top-level components.

2 Phase 1 — Mainnet-Grade Hardening (Milestone 1)

The first phase removes the operational and trust assumptions that are acceptable on testnet but not on mainnet. It targets the Layer 2 attestation path and the configurability of the guardrails.

2.1 Hardware-Backed Attester Keys

Today the `oracle-watch` attester signs snapshots with an Ed25519 key supplied through an environment variable. For mainnet we move signing behind an HSM / KMS interface, so the private key never resides in application memory.

```rust
pub trait AttesterSigner {
    fn public_key(&self) -> Address;
    fn sign(&self, payload: &[u8]) -> Signature;   // delegates to HSM/KMS
}
```

The daemon depends only on the trait; the environment-variable signer remains for local development, while a cloud-KMS signer (and a documented key-rotation procedure) is used in production.

2.2 Multi-Attester Quorum

The registry currently accepts a single whitelisted attester per write. We generalize it to an m-of-n quorum so that no single attester can move a liquidity snapshot on its own.

```rust
pub struct AttesterSet {
    pub members: Vec<Address>,   // n whitelisted attesters
    pub threshold: u32,          // m required signatures (m <= n)
}

// write_snapshot now verifies >= threshold valid signatures
// over the same (asset, volume, trades, timestamp) tuple.
```

This converts the Layer 2 trust vector from "trust one signer" to "trust that fewer than m of n are compromised," a materially stronger posture, while keeping Layer 1 fully trustless and unchanged.

2.3 Per-Asset Configuration

The library's thresholds are global today. Different assets have different volatility and liquidity profiles, so we add per-asset overrides resolved at call time, falling back to the global default when no override exists.

```rust
// Resolve: per-asset override if present, else global default.
pub fn config_for(asset: &Asset, base: &SafeOracleConfig)
    -> SafeOracleConfig;
```

Milestone 1 deliverables

- HSM/KMS `AttesterSigner` integration + documented key rotation. *Measure: oracle-watch submits a snapshot signed by a KMS-held key on testnet (verifiable tx).*
- m-of-n multi-attester quorum in `liquidity-registry`. *Measure: a snapshot written only after m signatures; a sub-threshold attempt rejected on-chain (two verifiable txs).*
- Per-asset threshold configuration with global fallback. *Measure: tests showing two assets validated under different thresholds in one call path.*
- Expanded adversarial + property test suite for the above. *Measure: test count increase reported by `cargo test -workspace`.*

3 Phase 2 — Aggregation, Integration & SDK (Milestone 2)

The second phase deepens the price math, proves the library against a real ecosystem protocol, and makes adoption easy through an SDK and documentation.

3.1 Time-Weighted Aggregation Guardrails

The current deviation check compares two consecutive prices. Following guidance from the Reflector lead developer, we add configurable time-weighted aggregation *on top of* the two-point check—computed consumer-side from the multi-record SEP-40 `prices(asset, records)` call (Reflector itself exposes no native `twap` entrypoint). Integrators choose the mode that fits their risk model.

```rust
pub enum AggregationMode { TwoPoint, Twap, Sma, Ema }

pub struct TwapConfig {
    pub mode: AggregationMode,
    pub window_records: u32,    // how many prices() records to weight
    pub max_deviation_bps: u32 // deviation of spot vs aggregate
}
```

A spot price that deviates from its time-weighted aggregate beyond the configured bound is rejected—smoothing single-tick manipulation that a raw two-point comparison could, in edge cases, straddle.

3.2 Reference Integration with an Integration-List Protocol

We build a complete, documented reference integration showing `safe-oracle` guarding a price-gated action in a protocol drawn from the SCF Integration List (the lending / CDP / swap-routing cohort—e.g. Blend pools, Soroswap routing, or DeFindex strategies—is the natural fit, since these act financially on oracle output). This replaces the `mock-lending` fixture with a realistic adoption example.

Figure 2: Reference integration: an ecosystem protocol gates its price-sensitive action through safe-oracle before reading Reflector.

3.3 Public SDK and Documentation Site

Adoption requires more than a crate. We publish helper bindings and a documentation site covering the standard, the integration guide, threshold-calibration guidance, and worked examples, so a team can integrate without reading the source.

Milestone 2 deliverables

- Configurable TWAP / SMA / EMA aggregation guardrail. *Measure: tests showing a manipulation that passes two-point but is rejected under TWAP, run through `lastprice()`.*
- Reference integration with one SCF Integration-List protocol on testnet. *Measure: a guarded price-gated action recorded on testnet (verifiable tx), plus a rejected adversarial attempt.*
- Public documentation site (standard + integration guide + examples). *Measure: site live at a public URL with the integration guide published.*
- SDK / helper bindings published. *Measure: a released, versioned package consuming the guardrail verdict.*

4 Phase 3 — Mainnet Launch (Milestone 3)

The final phase puts the hardened, deepened library into production against live Reflector, with the monitoring and metrics a production safety component requires.

4.1 Mainnet Deployment

We deploy `oracle-validator` and `liquidity-registry` to Stellar mainnet with strict production defaults (the values documented throughout the whitepaper: 2000 bps deviation, 300/900 s staleness, $10,000 liquidity floor), wired to the mainnet Reflector contract. The deviation guardrail is validated against the real mainnet Reflector `prices` history rather than a mock.

4.2 24/7 Monitoring and Production Metrics

The `oracle-watch` daemon runs continuously against mainnet SDEX with an incident sink (PagerDuty / Opsgenie via the existing trait), a documented runbook for the circuit-breaker manual override, and a disaster-recovery procedure for attester-key compromise. We publish a live metrics surface (snapshots written, guardrail outcomes, circuit-breaker events) so adopters can see the system working.

4.3 Audit and Launch Readiness

At testnet launch the project becomes eligible to apply for credits from the SDF Audit Bank; the independent audit (whose cost is *not* part of this budget) is completed on the path to mainnet, and its findings are remediated before the mainnet tag.

Milestone 3 deliverables (Mainnet launch)

- `oracle-validator` + `liquidity-registry` live on Stellar mainnet. *Measure: published mainnet contract addresses + a verifiable invocation tx.*
- Live Reflector integration validated on mainnet. *Measure: a guardrail outcome produced against the real mainnet Reflector feed (verifiable tx).*
- 24/7 oracle-watch monitoring with incident alerting and runbook. *Measure: continuous snapshot submissions over a sustained window + a published runbook.*
- Public production metrics surface. *Measure: live dashboard URL showing guardrail outcomes and snapshot counts.*

5 Summary

Throughout, Layer 1 remains fully trustless and unchanged in posture; the roadmap hardens the opt-in trust path, deepens the price math, proves adoption against a real ecosystem protocol, and ships to mainnet. The end state is a reusable, audited, monitored oracle-safety layer that any Stellar DeFi protocol can adopt to close the integrator-side gap that the February 2026 incident made concrete.

"We don't replace oracles. We validate them."

— safe-oracle

Table 1: Roadmap overview — features mapped to milestones

Milestone	**Theme**	**Key technical change**
1	Hardening	HSM/KMS attester signing; m-of-n attester quorum; per-asset thresholds; expanded test suite
2	Integration	TWAP/SMA/EMA aggregation guardrail; reference integration with an Integration-List protocol; public SDK + docs site
3	Mainnet	Mainnet deployment of validator + registry; live Reflector validation; 24/7 monitoring; public production metrics